

We Found the Replication Origin in *E. Coli* **BUT...**

```
aatgatgatgacgtcaaaaggatccggataaaacatggtgattgcctcgcataacgcggtatgaaaatggattgaagccccgggcccgtggattctactcaactttgtcggcttgagaaagacc  
tgggatcctgggtattaaaaagaagatctatttatttagagatctgttctatttgtgatctc  
ttattaggatcgcaactgccctgtggataacaaggatccggcttttaagatcaacaacctggaag  
aatgatcattaaactgtgaatgatcggtgatcctggaccgtataagctgggatcagaatgagggg  
gttatacacaaactcaaaaactgaacaacagttgttctttggataactaccgggttgatccaagc  
ttcctgacagagttatccacagtagatcgcacgatctgtatacttatttgagtaaa  
ttaaccacgatcccagccattcttctgccggatcttccggaatgtcgtgatcaagaatgt  
tgatcttcagtg
```

But there are **no** frequent 9-mers (that appear three or more times) in this region of *E. coli*!

STOP: Any ideas? Should we give up?

Accounting for Point Mutations

```
aatgatgatgacgtcaaaaggatccggataaaacatggtgattgcctcgcataacgcggtg  
tgaaaatggattgaagcccgggcccgtggattctactcaactttgtcggcttgagaaagacc  
tgggatcctgggtattaaaaagaagatctatttatttagagatctgttctattgtgatctc  
ttattaggatcgactgcccTGTGGATAAcaaggatccggcttttaagatcaacaacctgg  
aaaggatcattaactgtgaatgatcggatgatcctggaccgtataagctgggatcagaatga  
ggggTTATACACAactcaaaaactgaacaacagttgttctTTGGATAACtaccggttgatc  
caagcttcctgacagagTTATCCACAgtagatcgacgatctgtatacttatttgagtaaa  
ttaaccacgatcccagccattcttctgccggatcttccggaatgtcgtgatcaagaatgt  
tgatcttcagtg
```

Frequent 9-mers (with 1 Mismatch and Reverse Complements) in putative *ori* of *E. coli*

How Do We Find Regulatory Motifs in DNA?

Greedy and Randomized Algorithms

Phillip Compeau and Pavel Pevzner.

Bioinformatics Algorithms: An Active Learning Approach

©2013 by Compeau and Pevzner. All rights reserved.

Outline

- From Implanted Patterns to Regulatory Motifs
 - Implanting Patterns into Strings
 - Do We Have a Clock Gene?
 - Implanted Motif Problem
- Motif Finding Problem
 - Scoring Motifs
 - Reformulating Motif Finding Problem
- From Motifs to Profiles
 - Median String Problem
 - Greedy Motif Search
- How Rolling Dice Helps Us Find Regulatory Motifs
 - Randomized Motif Search
 - How do Bacteria Hibernate?
- Gibbs Sampling
 - Gibbs Sampler in Action
 - Pseudocounts

Outline

- **From Implanted Patterns to Regulatory Motifs**
 - **Implanting Patterns into Strings**
 - Do We Have a Clock Gene?
 - Implanted Motif Problem
 - Motif Finding Problem (Music Break)
 - Median String Problem
 - Greedy Motif Search
- How Rolling Dice Helps Us Find Regulatory Motifs
 - Randomized Motif Search
 - How do Bacteria Hibernate?
 - Gibbs Sampling
 - Pseudocounts

Generate Ten Random Sequences

```
atgaccgggatactgataccgtatttggcctaggcgtacacattagataaacgtatgaagtacgttagactcggcgccgccg  
acccctattttttgagcagatttagtgacctggaaaaaaaaatttgagtacaaaacttttccgaatactgggcataaggtaca  
tgagtatccctgggatgacttttgggaacactatagtgtctctccgatttttgaatatgtaggatcattcgccagggtccga  
gctgagaattggatgaccttgtaagtgttttccacgcaatcgcgaaaccaacgcggacccaaaggcaagaccgataaaggaga  
tcccttttgcggtaatgtgccgggaggctggttacgtagggaagccctaacggacttaatggcccacttagtccacttatag  
gtcaatcatgttcttgtgaatggatttttaactgagggcatagaccgcttggcgcacccaaattcagtgtgggcgagcgcaa  
cggttttggcccttgtagaggccccgtactgatggaaactttcaattatgagagagctaatttatcgcggtgcgtgttcat  
aacttgagttggtttcgaaaatgctctggggcacatacaagaggagtcttccttatcagttaatgctgtatgacactatgta  
ttggcccattggctaaaagcccaacttgacaaatggaagatagaatccttgcattttcaacgtatgccgaaccgaaagggaag  
ctggtgagcaacgacagattcttacgtgcattagctcgcttccggggatctaatagcacgaagcttctgggtactgatagca
```

Implant a Pattern **AAAAAAAAAGGGGGGGG** at Randomly Chosen Positions

```
atgaccgggatactgatAAAAAAAAAGGGGGGGG ggcgtacacattagataaacgtatgaagtacgttagactcggcgccgccg  
acccctatTTTTTgagcagatttagtgacctggaaaaaaatttgagtacaaaactTTTccgaataAAAAAAAAAGGGGGGGG a  
tgagtatccctgggatgacttAAAAAAAAAGGGGGGGG tgctctcccgattTTTgaatatgtaggatcattcgccagggtccga  
gctgagaattggatgAAAAAAAAAGGGGGGGG tccacgcaatcgcgaaaccaacgcggacccaaaggcaagaccgataaaggaga  
tccTTTTgcggtaatgtgccgggaggctggttacgtagggaagccctaacggacttaatAAAAAAAAAGGGGGGGG cttatag  
gtcaatcatgttcttTgtgaatggatttAAAAAAAAAGGGGGGGG gaccgcttggcgcacccaaattcagtgtgggcgagcgcaa  
cggTTTTggcccttgtagaggcccccgTAAAAAAAAAGGGGGGGG caattatgagagagctaattctatcgcgTgcgtgttcat  
aacttgagttAAAAAAAAAGGGGGGGG ctggggcacatacaagaggagtcttccttatcagttaatgctgtatgacactatgta  
ttggcccatTggctaaaagcccaacttgacaaatggaagatagaatccttgcatAAAAAAAAAGGGGGGGG accgaaagggaag  
ctggtgagcaacgacagattcttacgtgcattagctcgcttccggggatctaatagcacgaagcttAAAAAAAAAGGGGGGGG a
```

Where Are the Implanted Patterns Hiding?

```
atgaccgggatactgataaaaaaagggggggggcgctacacattagataaacgtatgaagtacgttagactcggcgccgccg
acccctatTTTTTgagcagatttagtgacctggaaaaaaatttgagtacaaaactTTTccgaataaaaaaaaaggggggga
tgagtatccctgggatgacttaaaaaaaggggggggtgctctcccgattTTTgaatatgtaggatcattcgccagggtccga
gctgagaattggatgaaaaaaaaggggggggtccacgcaatcgcgaaaccaacgcggacccaaaggcaagaccgataaaggaga
tccctTTTtgcggtaatgtgccgggaggctggttacgtagggaagccctaacggacttaataaaaaaaaagggggggcTTtag
gtcaatcatgttcttTgtgaatggatttaaaaaaaggggggggaccgcttggcgcacccaaattcagtgTgggcgagcgcaa
cggtTTTtggcccttgttagaggccccgtaaaaaaaggggggggcaattatgagagagctaatttatcgcgTgcgtgttcat
aacttgagttaaaaaaagggggggctggggcacatacaagaggagtcttccTTtatcagttaatgctgtatgacactatgta
ttggcccattggctaaaagcccaacttgacaaatggaagatagaatcTTtgcataaaaaaaggggggggaccgaaagggaag
ctggtgagcaacgacagattcttacgtgcattagctcgcttccggggatctaatagcacgaagcttaaaaaaaggggggga
```

The implanted patterns can be found by the Frequent Words algorithm.

Implant a Pattern **AAAAAAGGGGGGGG** with 4 Random Mutations at Random Positions

```
atgaccgggatactgatAgAAgAAAGGttGGGggcgtacacattagataaacgtatgaagtacgttagactcggcgccgccg  
acccctatTTTTTgagcagatttagtgacctggaaaaaaatttgagtacaaaactTTTccgaatacAAtAAAAcGGcGGGa  
tgagtatccctgggatgacttAAAAtAAtGGaGtGGtgctctcccgattTTTgaatatgtaggatcattcgccagggtccga  
gctgagaattggatgcAAAAAAGGG attGtccacgcaatcgcgaaacacgcggacccaaaggcaagaccgataaaggaga  
tccctTTTgcggtaatgtgccgggaggctggttacgtagggaagccctaacggacttaatAtAAtAAAGGaaGGGcttata  
gtcaatcatgttcttTgtgaatggatttAAcAAtAAGGGctGGgaccgcttggcgcacccaaattcagtgtgggcgagcgcaa  
cggTTTTggcccttgtagaggccccgtAtAAAcAAGGaGGGccaattatgagagagctaatttatcgcgTgcgtgttcat  
aacttgagttAAAAAA tAGGGaGccctggggcacatacaagaggagtcttccttatcagttaatgctgtatgacactatgta  
ttggcccatTggctaaaagcccaacttgacaaatggaagatagaatccttgcatActAAAAAGG aGcGGaccgaaagggaag  
ctggtgagcaacgacagattcttacgtgcattagctcgcttccggggatctaatagcacgaagcttActAAAAAGG aGcGGa
```

$A(k,d)$ -motif: a k -mer that appears in each sequence with at most d mismatches. **AAAAAAGGGGGGGG** is a **(15,4)-motif**

Now Where are the Implanted Patterns?

```
atgaccgggatactgatagaagaaaggttgggggctacacattagataaacgtatgaagtacgttagactcggcgccgccg  
acccctatTTTTTgagcagatttagtgacctggaaaaaaatttgagtacaaaactTTTccgaatacaataaaacggcgga  
tgagtatccctgggatgacttaaaataatggagtgggtgctctcccgattTTTgaatatgtaggatcattcgccagggtccga  
gctgagaattggatgcaaaaaaagggttgtccacgcaatcgcgaaaccaacgcggacccaaaggcaagaccgataaaggaga  
tccctTTTgcggtaatgtgccgggaggctggttacgtagggaagccctaacggacttaataataaaaggaagggttatag  
gtcaatcatgttcttTgtgaatggatttaacaataagggtgggaccgcttggcgcacccaaattcagtgtgggcgagcgcaa  
cggtTTTggcccttgtagaggccccgtataaacaaggaggggccaattatgagagagctaattctatcgcggtgcgtgttcat  
aacttgagttaaaaaataggagccctggggcacatacaagaggagtcttcttatcagttaatgctgtatgacactatgta  
ttggcccatTggctaaaagcccaacttgacaaatggaagatagaatccttgcatactaaaaaggagcggaccgaaagggaag  
ctggtgagcaacgacagattcttacgtgcattagctcgcttccggggatctaatagcacgaagcttactaaaaaggagcgga
```

Would the Frequent Words algorithm find this hidden message?

Why Would a Biologist Care?

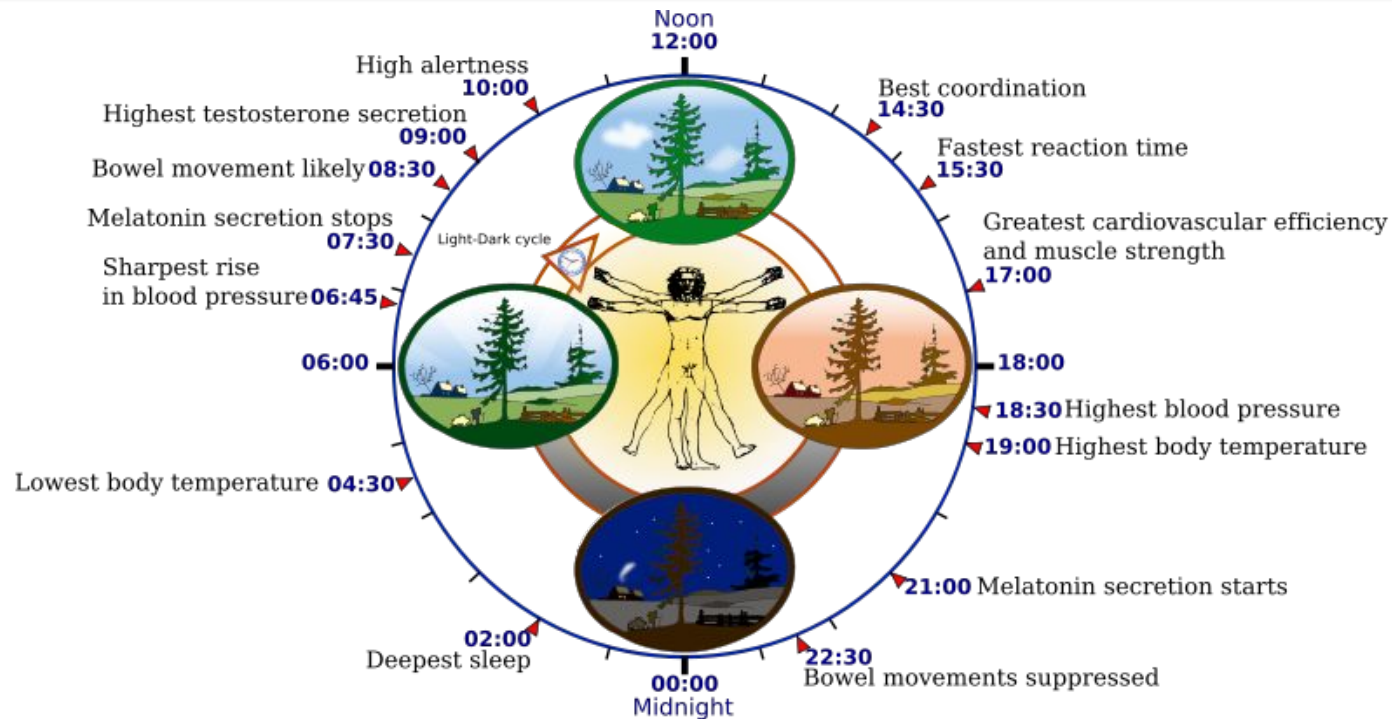


Outline

- **From Implanted Patterns to Regulatory Motifs**
 - Implanting Patterns into Strings
 - **Do We Have a Clock Gene?**
 - Implanted Motif Problem
 - Motif Finding Problem (Music Break)
 - Median String Problem
 - Greedy Motif Search
- How Rolling Dice Helps Us Find Regulatory Motifs
 - Randomized Motif Search
 - How do Bacteria Hibernate?
 - Gibbs Sampling
 - Pseudocounts

Circadian Clock

Our schedules are controlled by a molecular timekeeper called the **circadian clock**.



How does each cell in our body know when it should decrease or increase **gene expression** of circadian genes?

How do Plants Know Whether it is Day or Night?

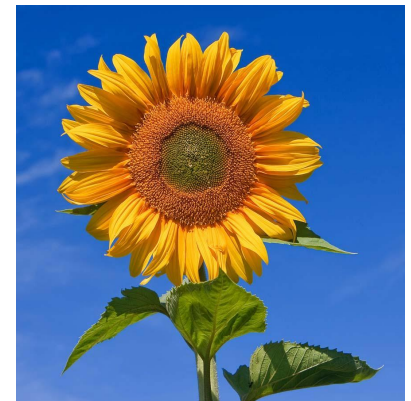
Plants need to **change gene expression** of 1000+ genes (photosynthesis, flowering, frost resistance, etc.) switching from day to night and vice-versa.



“Raat ki rani”
only blooms at night

Who are the “molecular managers” telling genes to switch expression on and off in billions of cells?

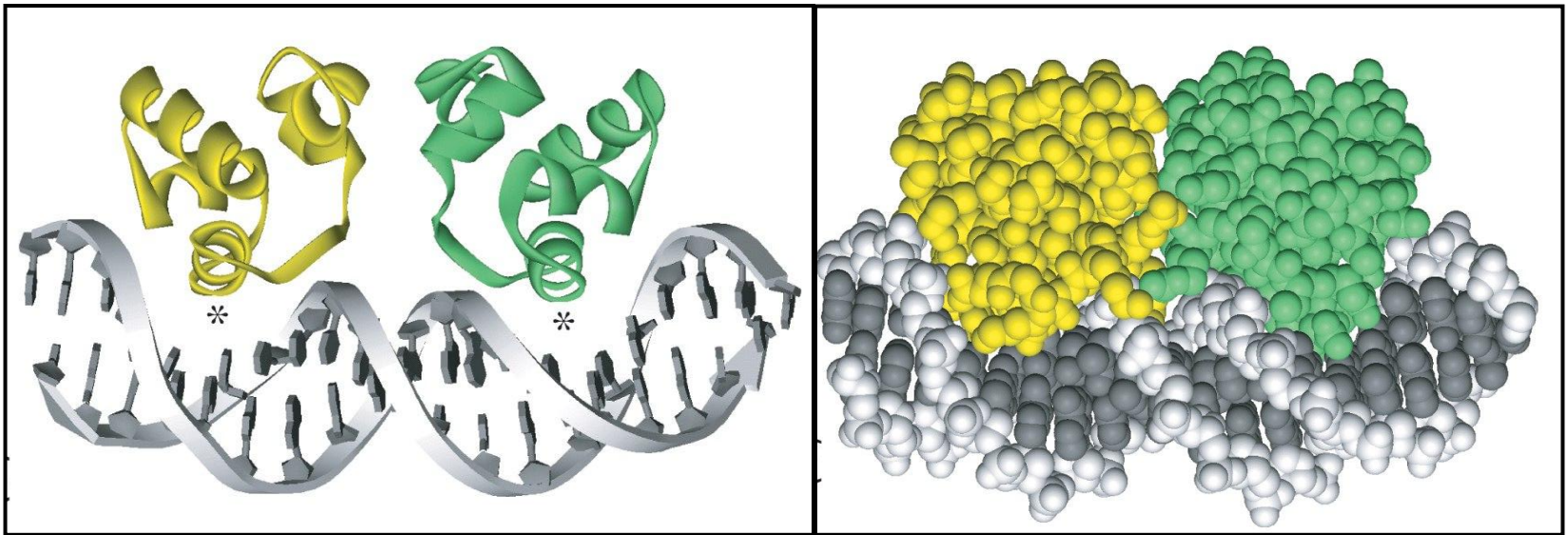
**Just 3 genes call the shots:
CCA1, LCY, and TOC1.**



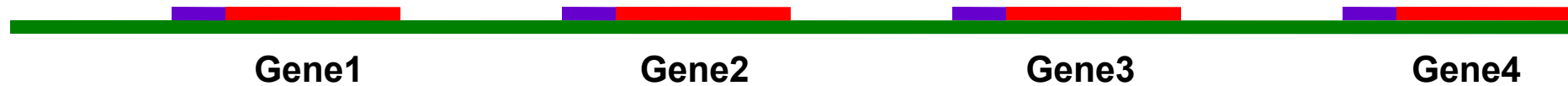
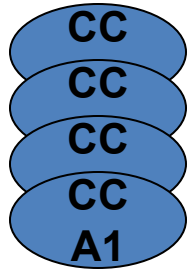
Sunflower
follows the sun

Transcription Factors and Their Binding Sites

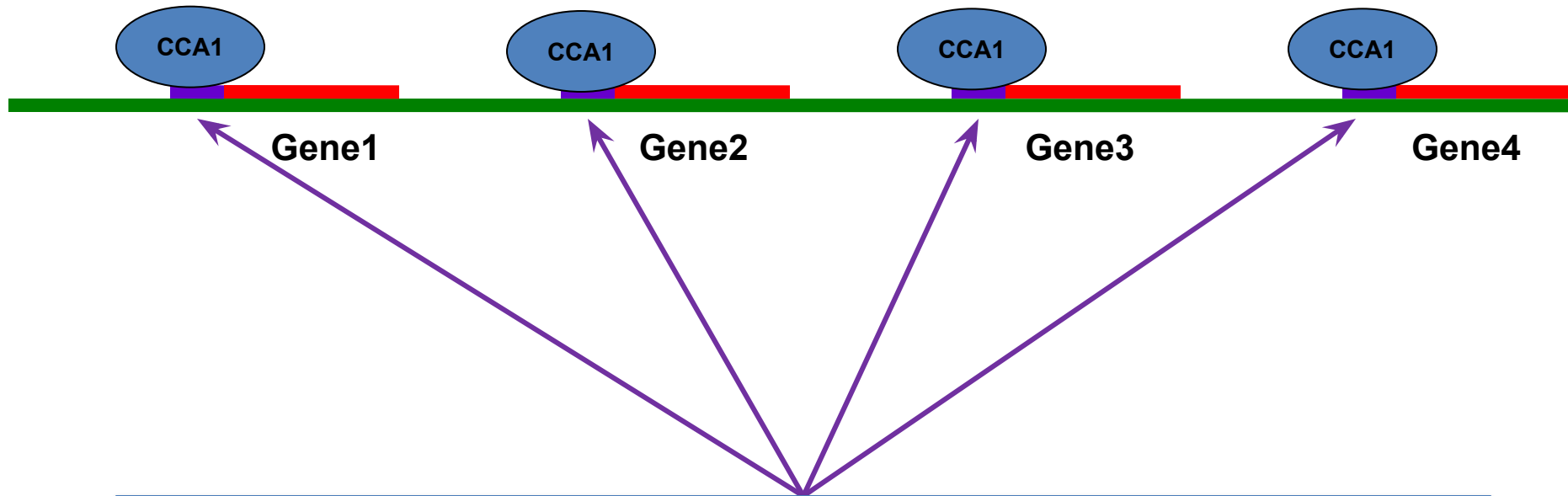
CCA1, LCY, and TOC1 are **regulatory proteins** (a.k.a. **transcription factors**) control other genes by binding to short DNA fragments (**transcription factor binding sites**) in the upstream regions of these genes.



Transcription Factors Bind to Their Binding Sites



How Does CCA1 Know Where to Bind?



There must be some hidden messages in these regions that tells CCA1 **WHERE** to bind.

Transcription Factor Binding Sites

cagt**ATAAGTCT**actgatgcaacctgactcatgacgaggaa

Gene1

agtcgactgactt**AACAATCT**cggatcgattcgtccgagga

Gene2

cgtcagctctgtcgggattcgcccggtatt**AAAAAGCT**ac

Gene3

accgtctcc**ACAAAACCT**gctcgtgcactgatgcaacctga

Gene4

The hidden messages (motif **AAAAAATCT**):
transcription factor binding sites of CCA1

Where are the Hidden Messages Hiding?

cagtataaagtctactgatgcaacctgactcatgacgaggaa

Gene1

agtcgactgacttaacaaatctcggatcgattcgtccgagga

Gene2

cgtcagctctgtcgggattcgccccgtattcaaaaaagcac

Gene3

accgtctccacaaaacctgctcgtgccactgatgcaacctga

Gene4

Outline

- **From Implanted Patterns to Regulatory Motifs**
 - Implanting Patterns into Strings
 - Do We Have a Clock Gene?
 - **Implanted Motif Problem**
 - Motif Finding Problem (Music Break)
 - Median String Problem
 - Greedy Motif Search
- How Rolling Dice Helps Us Find Regulatory Motifs
 - Randomized Motif Search
 - How do Bacteria Hibernate?
 - Gibbs Sampling
 - Pseudocounts

Randomly Implant **AAAAAAGGGGGGGG** with 4 Random Mutations at 4 Random Positions

```
atgaccgggatactgatAgAAgAAAGGttGGGggcgtacacattagataaacgtatgaagtacgttagactcggcgccgccg  
accctatTTTTTTgagcagatttagtgacctggaaaaaaatttgagtacaaaactTTTccgaatacAAtAAAAcGGcGGGa  
tgagtatccctgggatgacttAAAAtAAtGGaGtGGtgctctcccgattTTTTgaatatgtaggatcattcgccagggtccga  
gctgagaattggatgcAAAAAAAGGG attGtccacgcaatcgcgaaaccaacgcggacccaaaggcaagaccgataaaggaga  
tccTTTTTgcggtaatgtgccgggaggctggttacgtagggaagccctaacggacttaatAtAAtAAAGGaaGGGcttata  
gtcaatcatgttcttTgtgaatggatttAAcAAtAAGGGctGGgaccgcttggcgcacccaaattcagtgtgggcgagcgcaa  
cggTTTTTggcccttgtagaggcccccgtAtAAAcAAGGaGGGccaattatgagagagctaattctatcgcgtgcgtgttcat  
aacttgagttAAAAAA tAGGGaGccctggggcacatacaagaggagtcttccttatcagttaatgctgtatgacactatgta  
ttggcccatTggctaaaagcccaacttgacaaatggaagatagaatccttgcatActAAAAAGG aGcGGaccgaaagggaag  
ctggtgagcaacgacagattcttacgtgcattagctcgcttccggggatctaatagcacgaagcttActAAAAAGG aGcGGa
```

Frequent Words algorithm cannot find this (15,4) motif due to too many mutations between the hidden messages

Implanted Motifs Problem

Implanted Motif Problem. Finding (k,d) -motifs in a set of strings.

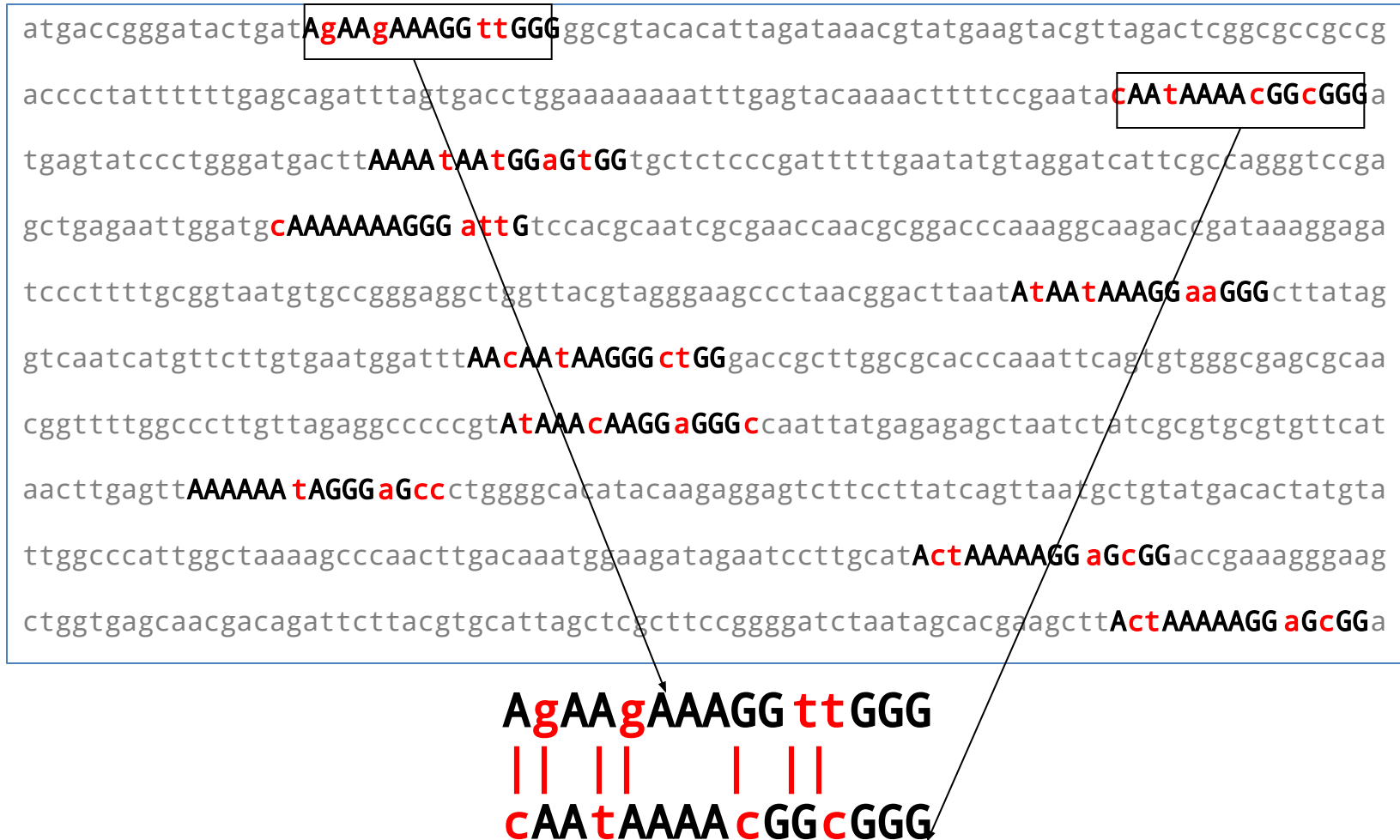
- **Input:** A set of strings Dna , and integers k (motif length) and d (maximal number of mismatches in a motif).
- **Output:** All (k,d) -motifs in Dna .

Should we explore ALL 4^k k -mers to solve the Implanted Motif Problem?

Finding Implanted Motifs by Pairwise Comparison

```
atgaccgggatactgatAgAAgAAAGGttGGGggcgtacacattagataaacgtatgaagtacgttagactcggcgccgccg
acccctatTTTTTgagcagatttagtgacctggaaaaaaatttgagtacaaaacttttccgaatacAAtAAAAcGGcGGGa
tgagtatccctgggatgacttAAAAtAAtGGaGtGGtgctctcccgatttttgaatatgtaggatcattcgccagggtccga
gctgagaattggatgcAAAAAAGGG attGtccacgcaatcggaaccaacgcggacccaaaggcaagaccgataaaggaga
tcccttttgcggtaatgtgccgggaggctggttacgtagggaagccctaacggacttaatAtAAtAAAGG aaGGGcttatag
gtcaatcatgttcttgtgaatggatttAAcAAtAAGGG ctGGgaccgcttggcgcacccaaattcagtgtgggcgagcgcaa
cggttttggcccttgtagaggcccccgAtAAAAcAAGGaGGGccaattatgagagagctaatactatcgcggtgcgtgttcat
aacttgagttAAAAA tAGGG aGccctggggcacatacaagaggagtcttccttatcagttaatgctgtatgacactatgta
ttggcccataggctaaaagcccaacttgacaaatggaagatagaatccttgcAtActAAAAAGG aGcGGaccgaaagggaag
ctggtgagcaacgacagattcttacgtgcattagctcgcttccggggatctaatagcacgaagcttActAAAAAGG aGcGGa
```

Finding Implanted Motifs by Pairwise Comparison



Why Pairwise Comparison Won't Work

AgAA**g**AAAGG**tt**GGG *Implanted instance 1*

| | | | **4 mismatches**

AAAAAAAAAGGGGGGGG **+** *Pattern*

| | | | **4 mismatches**

cAA**t**AAAA**c**GGGGGG**c** *Implanted instance 2*

The implanted instances have up to **4 + 4 = 8 mutations** – they don't even look like similar strings!

Resorting to Motif Enumeration Instead

MotifEnumeration(Dna, k, d)

for each k -mer a

for each k -mer a' in DNA differing from a by at most d mutations

if a' is a (k,d) -mer

output a'

Would this simple (albeit slow) algorithm work for finding **real** biological motifs?

MotifEnumeration assumes that **EACH** sequence has a k -mer similar to an (unknown) implanted pattern. This condition does not hold for noisy biological datasets.

Outline

- **From Implanted Patterns to Regulatory Motifs**
 - Implanting Patterns into Strings
 - Do We Have a Clock Gene?
 - Implanted Motif Problem
 - **Motif Finding Problem**
 - Median String Problem
 - Greedy Motif Search
- How Rolling Dice Helps Us Find Regulatory Motifs
 - Randomized Motif Search
 - How do Bacteria Hibernate?
 - Gibbs Sampling
 - Pseudocounts

From real Motifs to a Consensus String

NfKB Motifs

What does NfKB do?

*How is it related to
Covid-19 infection ?*

T	C	G	G	G	G	g	T	T	T	t	t
c	C	G	G	t	G	A	c	T	T	a	C
a	C	G	G	G	G	A	T	T	T	t	C
T	t	G	G	G	G	A	c	T	T	t	t
a	a	G	G	G	G	A	c	T	T	C	C
T	t	G	G	G	G	A	c	T	T	C	C
T	C	G	G	G	G	A	T	T	c	a	t
T	C	G	G	G	G	A	T	T	c	C	t
T	a	G	G	G	G	A	a	c	T	a	C
T	C	G	G	G	t	A	T	a	a	C	C

From **Real** Motifs to a Consensus String

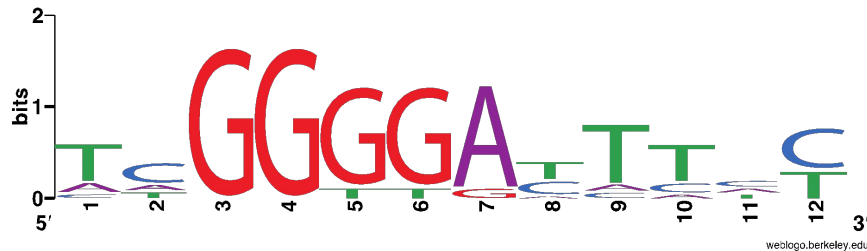
T	C	G	G	G	G	g	T	T	T	t	t
c	C	G	G	t	G	A	c	T	T	a	C
a	C	G	G	G	G	A	T	T	T	t	C
T	t	G	G	G	G	A	c	T	T	t	t
a	a	G	G	G	G	A	c	T	T	C	C
T	t	G	G	G	G	A	c	T	T	C	C
T	C	G	G	G	G	A	T	T	c	a	t
T	C	G	G	G	G	A	T	T	c	C	t
T	a	G	G	G	G	A	a	c	T	a	C
T	C	G	G	G	t	A	T	a	a	C	C

From **Real** Motifs to a Consensus String

NfKB Motifs

T	C	G	G	G	G	g	T	T	T	t	t
c	C	G	G	t	G	A	c	T	T	a	C
a	C	G	G	G	G	A	T	T	T	t	C
T	t	G	G	G	G	A	c	T	T	t	t
a	a	G	G	G	G	A	c	T	T	C	C
T	t	G	G	G	G	A	c	T	T	C	C
T	C	G	G	G	G	A	T	T	c	a	t
T	C	G	G	G	G	A	T	T	c	C	t
T	a	G	G	G	G	A	a	c	T	a	C
T	C	G	G	G	t	A	T	a	a	C	C

Motif Logo



From Motifs to a Consensus String

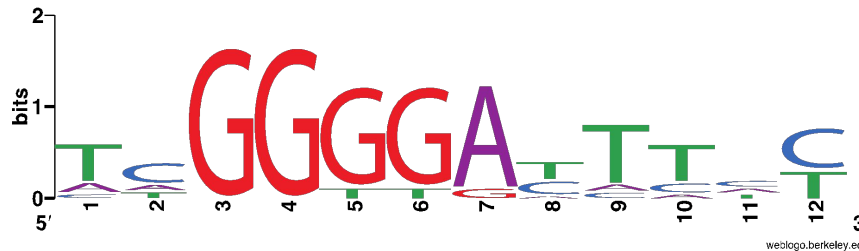
Motifs

T	C	G	G	G	G	g	T	T	T	t	t
c	C	G	G	t	G	A	c	T	T	a	C
a	C	G	G	G	G	A	T	T	T	t	C
T	t	G	G	G	G	A	c	T	T	t	t
a	a	G	G	G	G	A	c	T	T	C	C
T	t	G	G	G	G	A	c	T	T	C	C
T	C	G	G	G	G	A	T	T	c	a	t
T	C	G	G	G	G	A	T	T	c	C	t
T	a	G	G	G	G	A	a	c	T	a	C
T	C	G	G	G	t	A	T	a	a	C	C

Consensus (Motifs)

T	C	G	G	G	G	A	T	T	T	C
C										

Motif Logo



From Motifs to a Consensus String

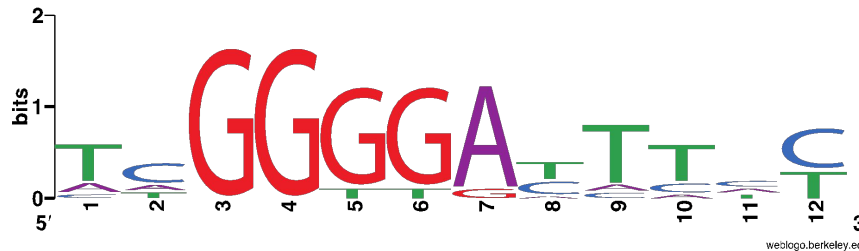
Motifs

T	C	G	G	G	G	g	T	T	T	t	t
c	C	G	G	t	G	A	c	T	T	a	C
a	C	G	G	G	G	A	T	T	T	t	C
T	t	G	G	G	G	A	c	T	T	t	t
a	a	G	G	G	G	A	c	T	T	C	C
T	t	G	G	G	G	A	c	T	T	C	C
T	C	G	G	G	G	A	T	T	c	a	t
T	C	G	G	G	G	A	T	T	c	C	t
T	a	G	G	G	G	A	a	c	T	a	C
T	C	G	G	G	t	A	T	a	a	C	C

Consensus (Motifs)

T	C	G	G	G	G	A	T	T	T	T	C
C											

Motif Logo



Score(Motifs)

$$3 + 4 + 0 + 0 + 1 + 1 + 5 + 2 + 3 + 6 + 4 = 30$$

Score(Motifs): # of lowercase letters in motif matrix

The Motif Finding Problem

Motif Finding Problem : *Given a collection of strings, find a set of k -mers, one from each string, that minimizes the score of the resulting motif.*

- **Input:** A collection of strings Dna and an integer k .
- **Output:** A collection $Motifs$ of k -mers, one from each string in Dna , minimizing $Score(Motifs)$ among all possible choices of k -mers.

Brute Force Motif Search

Brute Force Motif Search : Compute score of every possible choice of k -mers in Dna .

Brute Force Motif Search

Brute Force Motif Search : Compute score of every possible choice of k -mers in Dna .

STOP and Think: What is this approach's runtime?

Brute Force Motif Search

Brute Force Motif Search : Compute score of every possible choice of k -mers in Dna .

STOP and Think: What is this approach's runtime?

k-mer

AAAAAAAAAAAAA AAAAAAAAAATTTCTTTTCAAAAAAAAAAAAAAAAAAGGGGGGGGGGG GGGGGGGGGGGG
CCCCCCCCCCCCCCCCCCCCCCCCCCTTCCTTTTTAAAAAAAAAGGGGGGGGGGG GGGGGGGGGGGG
CCCCAAAAAACCCCAAACCCCAACCCCAAAAAATCTTTTCTGGGGCCCCCG CCCCCGGGGG

..

.. t DNA sequences

■ ■ ■ ● ●

[illegible]

$n-k+1$ * $n-k+1$ * $n-k+1$ * $n-k+1$ * $n-k+1$ t times = $(n-k+1)^t$: Extracting motifs

AAAAAAAAAA

CCCCCCCCCCCC

CCCCAAAA

.. t DNA sequences

AAAAAACTCT

k*t; comparisons per extracted motifs for scoring

Brute Force Motif Search

Brute Force Motif Search : Compute score of every possible choice of k -mers in Dna .

STOP and Think: What is this approach's runtime?

Say that Dna consists of t strings of length n :

- There are $(n - k + 1)^t$ different choices of *Motifs*
- Scoring the matrix requires $k * t$ steps.

Brute Force Motif Search

Brute Force Motif Search : Compute score of every possible choice of k -mers in Dna .

STOP and Think: What is this approach's runtime?

Say that Dna consists of t strings of length n :

- There are $(n - k + 1)^t$ different choices of *Motifs*
- Scoring the matrix requires $k * t$ steps.

Overall runtime $\sim O(n^t * k * t)$ – **too slow!**

Changing Our Perspective

<i>Motifs</i>	T	C	G	G	G	G	g	T	T	T	T	t
	c	C	G	G	t	G	A	c	T	T	a	C
	a	C	G	G	G	G	A	T	T	T	t	C
	T	t	G	G	G	G	A	c	T	T	t	t
	a	a	G	G	G	G	A	c	T	T	C	C
	T	t	G	G	G	G	A	c	T	T	C	C
	T	C	G	G	G	G	A	T	T	c	a	t
	T	C	G	G	G	G	A	T	T	c	C	t
	T	a	G	G	G	G	A	a	c	T	a	C
	T	C	G	G	G	t	A	T	a	a	C	C
<i>Consensus (Motifs)</i>	T C	C	G	G	G	G	A	T	T	T	C	



Changing Our Perspective

Motifs

T	C	G	G	G	G	g	T	T	T	T	t
c	C	G	G	t	G	A	c	T	T	a	C
a	C	G	G	G	G	A	T	T	T	t	C
T	t	G	G	G	G	A	c	T	T	t	t
a	a	G	G	G	G	A	c	T	T	C	C
T	t	G	G	G	G	A	c	T	T	C	C
T	C	G	G	G	G	A	T	T	c	a	t
T	C	G	G	G	G	A	T	T	c	C	t
T	a	G	G	G	G	A	a	c	T	a	C
T	C	G	G	G	t	A	T	a	a	C	C

Consensus (Motifs)

T	C	G	G	G	G	A	T	T	T	C
C										



Changing Our Perspective

<i>Motifs</i>	T	C	G	G	G	G	g	T	T	T	T	t
	c	C	G	G	t	G	A	c	T	T	a	C
	a	C	G	G	G	G	A	T	T	T	t	C
	T	t	G	G	G	G	A	c	T	T	t	t
	a	a	G	G	G	G	A	c	T	T	C	C
	T	t	G	G	G	G	A	c	T	T	C	C
	T	C	G	G	G	G	A	T	T	c	a	t
	T	C	G	G	G	G	A	T	T	c	C	t
	T	a	G	G	G	G	A	a	c	T	a	C
	T	C	G	G	G	t	A	T	a	a	C	C
<i>Consensus (Motifs)</i>	T	C	G	G	G	G	A	T	T	T	C	C
<i>Score (Motifs)</i>	3 + 4 + 0 + 0 + 1 + 1 + 5 + 2 + 3 + 6 + 4 = 30											

Changing Our Perspective

<i>Motifs</i>	T	C	G	G	G	G	g	T	T	T	T	t	3
	c	C	G	G	t	G	A	c	T	T	a	C	4
	a	C	G	G	G	G	A	T	T	T	t	C	2
	T	t	G	G	G	G	A	c	T	T	t	t	4
	a	a	G	G	G	G	A	c	T	T	C	C	3
	T	t	G	G	G	G	A	c	T	T	C	C	2
	T	C	G	G	G	G	A	T	T	c	a	t	3
	T	C	G	G	G	G	A	T	T	c	C	t	2
	T	a	G	G	G	G	A	a	c	T	a	C	4
	T	C	G	G	G	t	A	T	a	a	C	C	3
<i>Consensus (Motifs)</i>	T	C	G	G	G	G	A	T	T	T	T	C	
	C												
<i>Score(Motifs)</i>	3 + 4 + 0 + 0 + 1 + 1 + 5 + 2 + 3 + 6 + 4 = 30												



Changing Our Perspective

<i>Motifs</i>	T	C	G	G	G	G	g	T	T	T	T	t	3
	c	C	G	G	t	G	A	c	T	T	a	C	4
	a	C	G	G	G	G	A	T	T	T	t	C	2
	T	t	G	G	G	G	A	c	T	T	t	t	4
	a	a	G	G	G	G	A	c	T	T	C	C	3
	T	t	G	G	G	G	A	c	T	T	C	C	2
	T	C	G	G	G	G	A	T	T	c	a	t	3
	T	C	G	G	G	G	A	T	T	c	C	t	2
	T	a	G	G	G	G	A	a	c	T	a	C	4
	T	C	G	G	G	t	A	T	a	a	C	C	3
<i>Consensus (Motifs)</i>	T	C	G	G	G	G	A	T	T	T	T	C	
	C												
<i>Score (Motifs)</i>	3 + 4 + 0 + 0 + 1 + 1 + 5 + 2 + 3 + 6 + 4 = 30												

Number of lowercase symbols in a row = Hamming distance between that row and *Consensus(Motifs)*.

Changing Our Perspective

<i>Motifs</i>	T	C	G	G	G	G	g	T	T	T	T	t	3
	c	C	G	G	t	G	A	c	T	T	a	C	4
	a	C	G	G	G	G	A	T	T	T	t	C	2
	T	t	G	G	G	G	A	c	T	T	t	t	4
	a	a	G	G	G	G	A	c	T	T	C	C	3
	T	t	G	G	G	G	A	c	T	T	C	C	2
	T	C	G	G	G	G	A	T	T	c	a	t	3
	T	C	G	G	G	G	A	T	T	c	C	t	2
	T	a	G	G	G	G	A	a	c	T	a	C	4
	T	C	G	G	G	t	A	T	a	a	C	C	3
<i>Consensus (Motifs)</i>	T	C	G	G	G	G	A	T	T	T	T	C	
	C												
<i>Score (Motifs)</i>	3 + 4 + 0 + 0 + 1 + 1 + 5 + 2 + 3 + 6 + 4 = 30												

Number of lowercase symbols in a row = Hamming distance between that row and *Consensus(Motifs)*.

$$\text{Score}(\text{Motifs}) = d(\text{Consensus}(\text{Motifs}), \text{Motifs})$$

The Equivalent Motif Finding Problem

Equivalent Motif Finding Problem : *Given a collection of strings, find a pattern and a collection of k -mers (one from each string) that minimizes the distance between all possible patterns and all possible collections of k -mers.*

- **Input:** A collection of strings Dna and an integer k .
- **Output:** A k -mer *Pattern* and a collection of k -mers *Motifs*, one from each string in Dna , minimizing $d(Pattern, Motifs)$ among all possible choices of *Pattern* and *Motifs*.

The Equivalent Motif Finding Problem

Equivalent Motif Finding Problem : *Given a collection of strings, find a pattern and a collection of k -mers (one from each string) that minimizes the distance between all possible patterns and all possible collections of k -mers.*

- **Input:** A collection of strings Dna and an integer k .
- **Output:** A k -mer *Pattern* and a collection of k -mers *Motifs*, one from each string in Dna , minimizing $d(Pattern, Motifs)$ among all possible choices of *Pattern* and *Motifs*.

Have We Just Made Our Task More Difficult?

Motif Finding Problem. Find a set of k -mers *Motifs*, one from each sequence in *Dna*, minimizing $\text{Score}(\text{Motifs})$.

Equivalent Motif Finding Problem. Find *k -mer Pattern* **AND** a set of k -mers *Motifs*, one from each sequence in *Dna*, minimizing $d(\text{Pattern}, \text{Motifs})$.

The key insight for solving the Equivalent Motif Finding Problem:
given *Pattern*, we can quickly find *Motifs* minimizing $d(\text{Pattern}, \text{Motifs})$.



No need to explore all *Motifs* like in **Motif Finding Problem**!
Instead, the **Median String Problem** explores all k -mers *Pattern*

Outline

- **From Implanted Patterns to Regulatory Motifs**
 - Implanting Patterns into Strings
 - Do We Have a Clock Gene?
 - Implanted Motif Problem
 - Motif Finding Problem (Music Break)
 - **Median String Problem**
 - Greedy Motif Search
- How Rolling Dice Helps Us Find Regulatory Motifs
 - Randomized Motif Search
 - How do Bacteria Hibernate?
 - Gibbs Sampling
 - Pseudocounts

Distance between a k -mer and a (longer) String

Distance $d(\text{GATTCTCA}, \text{GCAAAGACGCTGACCAA}) = ?$

G	A	T	T	C	T	C	A									
G	C	A	A	A	G	A	C	G	C	T	G	A	C	C	A	A

Distance: 7

$d(\text{Pattern}, \text{String})$:

minimum distance between *Pattern* and all k -mers in *String*

Distance between a k -mer and a (longer) String

$$d(\text{GATTCTCA}, \text{GCAAAGACGCTGACCAA}) = ?$$

	G	A	T	T	C	T	C	A								
G	C	A	A	A	G	A	C	G	C	T	G	A	C	C	A	A

Distance: 7 6

$d(\text{Pattern}, \text{String})$:

minimum distance between *Pattern* and all k -mers in *String*

Distance between a k -mer and a (longer) String

$$d(\text{GATTCTCA}, \text{GCAAAGACGCTGACCAA}) = ?$$

Diagram illustrating the alignment of the 8-mer GATTCTCA (blue) with the 16-mer GCAAAGACGCTGACCAA (green). Red vertical lines indicate mismatches at positions 3, 5, 6, 7, 8, and 9.

			G	A	T	T	C	T	C	A					
G	C	A	A	A	G	A	C	G	C	T	G	A	C	C	A

Distance: 7 6 7

$d(\text{Pattern}, \text{String})$:

minimum distance between *Pattern* and all k -mers in *String*

Distance between a k -mer and a (longer) String

$$d(\text{GATTCTCA}, \text{GCAAAGACGCTGACCAA}) = ?$$

 G A T T C T C A
 | | | |
 G C A A A G A C G C T G A C C A A

Distance: 7 6 7 5

$d(\text{Pattern}, \text{String})$:

minimum distance between *Pattern* and all k -mers in *String*

Distance between a k -mer and a (longer) String

$$d(\text{GATTCTCA}, \text{GCAAAGACGCTGACCAA}) = ?$$

Diagram illustrating the alignment of the k -mer GATTCTCA with the string GCAAAGACGCTGACCAA. Red vertical lines indicate mismatches at positions 5 through 12.

				G	A	T	T	C	T	C	A						
G	C	A	A	A	G	A	C	G	C	T	G	A	C	C	A	A	

Distance: 7 6 7 5 **8**

$d(\text{Pattern}, \text{String})$:

minimum distance between *Pattern* and all k -mers in *String*

Distance between a k -mer and a (longer) String

$$d(\text{GATTCTCA}, \text{GCAAAGACGCTGACCAA}) = ?$$

 G A T T C T C A
 | | |
 G C A A A G A C G C T G A C C A A

Distance: 7 6 7 5 8 **3**

$d(\text{Pattern}, \text{String})$:

minimum distance between *Pattern* and all k -mers in *String*

Distance between a k -mer and a (longer) String

$$d(\text{GATTCTCA}, \text{GCAAAGACGCTGACCAA}) = ?$$

 G A T T C T C A
 | | | | | | | |
 G C A A A G A C G C T G A C C A A

Distance: 7 6 7 5 8 3 **8**

$d(\text{Pattern}, \text{String})$:

minimum distance between *Pattern* and all k -mers in *String*

Distance between a k -mer and a (longer) String

$$d(\text{GATTCTCA}, \text{GCAAAGACGCTGACCAA}) = ?$$

 G A T T C T C A
 | | | | | | |
 G C A A A G A C G C T G A C C A A

Distance: 7 6 7 5 8 3 8 **7**

$d(\text{Pattern}, \text{String})$:

minimum distance between *Pattern* and all k -mers in *String*

Distance between a k -mer and a (longer) String

$$d(\text{GATTCTCA}, \text{GCAAAGACGCTGACCAA}) = ?$$

 G A T T C T C A
 | | | |
 G C A A A G A C G C T G A C C A A

Distance: 7 6 7 5 8 3 8 7 **4**

$d(\text{Pattern}, \text{String})$:

minimum distance between *Pattern* and all k -mers in *String*

Distance between a k -mer and a (longer) String

$$d(\text{GATTCTCA}, \text{GCAAAGACGCTGACCAA}) = ?$$

										G	A	T	T	C	T	C	A
G	C	A	A	A	G	A	C	G	C	T	G	A	C	C	A	A	

Distance: 7 6 7 5 8 3 8 7 4 **6**

$d(\text{Pattern}, \text{String})$:

minimum distance between *Pattern* and all k -mers in *String*

Distance between a k -mer and a (longer) String

$$d(\text{GATTCTCA}, \text{GCAAAGACGCTGACCAA}) = 3$$

G A T T C T C A
| | |
G C A A A G A C G C T G A C C A A

Distance: 7 6 7 5 8 **3** 8 7 4 6

$d(\text{Pattern}, \text{String})$:

minimum distance between *Pattern* and all k -mers in *String*

Distance between a k -mer and a **Set** of Strings

Distance between a k -mer and a set of strings $Dna = \{Dna_1, \dots, Dna_t\}$:

$$d(k\text{-mer}, Dna) = \sum d(k\text{-mer}, Dna_i)$$

all strings in Dna

Pattern = AAA

ttaccttAAc 1

gAtAtctgtc 1

Acggcgttcg 2

ccctAAAgag 0

cgtcAgAggt 1

$$d(AAA, Dna) = 5$$

A **median string** for the set of strings Dna :

a k -mer minimizing distance $d(k\text{-mer}, Dna)$ over all possible k -mers.

Median String Problem

Median String Problem. Finding a median string.

- **Input:** A set of sequences *Dna* and an integer k .
- **Output:** A *k-mer* minimizing distance $d(k\text{-mer}, Dna)$ among all k -mers.

MedianString(*Dna*, k)

best-k-mer $\leftarrow$ AAA $\cdot \cdot \cdot$ AA

for each *k-mer* from AAA $\cdot \cdot \cdot$ AA to TTT $\cdot \cdot \cdot$ TT

if $d(k\text{-mer}, Dna) < \text{distance}(\text{best-k-mer}, Dna)$

best-k-mer $\leftarrow$ *k-mer*

return(*best-k-mer*)

Runtime: $4^k \cdot n \cdot t \cdot k$ (for *Dna* with t sequences of length n).

Motif Finding Problem

versus

Median String Problem

Runtime: $n^t \cdot k \cdot t$

Runtime: $4^k \cdot n \cdot t \cdot k$



Motif Finding Problem = Median String Problem

Motif Finding Problem

versus

Median String Problem

Runtime: $n^t \cdot k \cdot t$

Runtime: $4^k \cdot n \cdot t \cdot k$



Although **MedianString** is much faster than **BruteForceMotifSearch**, it is still slow for large k .

Outline

- **From Implanted Patterns to Regulatory Motifs**
 - Implanting Patterns into Strings
 - Do We Have a Clock Gene?
 - Implanted Motif Problem
 - Motif Finding Problem (Music Break)
 - Median String Problem
 - **Greedy Motif Search**
- How Rolling Dice Helps Us Find Regulatory Motifs
 - Randomized Motif Search
 - How do Bacteria Hibernate?
 - Gibbs Sampling
 - Pseudocounts

From Motifs to Profile

Motifs

T	C	G	G	G	G	g	T	T	T	t	t
c	C	G	G	t	G	A	c	T	T	a	C
a	C	G	G	G	G	A	T	T	T	t	C
T	t	G	G	G	G	A	c	T	T	t	t
a	a	G	G	G	G	A	c	T	T	C	C
T	t	G	G	G	G	A	c	T	T	C	C
T	C	G	G	G	G	A	T	T	c	a	t
T	C	G	G	G	G	A	T	T	c	C	t
T	a	G	G	G	G	A	a	c	T	a	C
T	C	G	G	G	T	A	T	a	a	C	C

Profile(Motifs)

A:	.2	.2	0	0	0	0	.9	.1	.1	.1	.3	0
C:	.1	.6	0	0	0	0	0	.4	.1	.2	.4	.6
G:	0	0	1	1	.9	.9	.1	0	0	0	0	0
T:	.7	.2	0	0	.1	.1	0	.5	.8	.7	.3	.4

frequency
of nucleotide
i in column *j*

T	C	G	G	G	G	A	C	T	T	C	C
---	---	---	---	---	---	---	---	---	---	---	---



Each column of a profile represents a biased 4-sided **die** with A, C, G, and T on its sides. Thus, a profile corresponds to *k* dice.

What is the probability that *k* rolls of such dice generate a given string ?

Scoring k -mers with a *Profile*

Given the following *Profile*:

A	1/2	7/8	3/8	0	1/8	0
C	1/8	0	1/2	5/8	3/8	0
T	1/8	1/8	0	0	1/4	7/8
G	1/4	0	1/8	3/8	1/4	1/8

The probability of the consensus string:

$$\Pr(\text{AAACCT} | \text{Profile}) = ???$$

Scoring k -mers with a *Profile*

Given the following *Profile*:

A	1/2	7/8	3/8	0	1/8	0
C	1/8	0	1/2	5/8	3/8	0
T	1/8	1/8	0	0	1/4	7/8
G	1/4	0	1/8	3/8	1/4	1/8

The probability of the consensus string:

$$\Pr(\text{AAACCT} | \text{Profile}) =$$

$$\frac{1}{2} \times \frac{7}{8} \times \frac{3}{8} \times \frac{5}{8} \times \frac{3}{8} \times \frac{7}{8} =$$
$$0.033646$$

Scoring k -mers with a *Profile*

Given the following *Profile*:

A	1/2	7/8	3/8	0	1/8	0
C	1/8	0	1/2	5/8	3/8	0
T	1/8	1/8	0	0	1/4	7/8
G	1/4	0	1/8	3/8	1/4	1/8

The probability of another string:

$$\Pr(\text{AAACCT} | \text{Profile}) =$$

$$1/2 \times 7/8 \times 3/8 \times 5/8 \times 3/8 \times 7/8 = 0.033646$$

$$\Pr(\text{ATACAG} | \text{Profile}) =$$

$$1/2 \times 1/8 \times 3/8 \times 5/8 \times 1/8 \times 1/8 = 0.001602$$

The closer k -mer is to the consensus string, the larger $\Pr(k\text{-mer} | \text{Profile})$ is.

What is the *Profile*-most probable 6-mer in
CTATAAACCTTACAT?

A	1/2	7/8	3/8	0	1/8	0
C	1/8	0	1/2	5/8	3/8	0
T	1/8	1/8	0	0	1/4	7/8
G	1/4	0	1/8	3/8	1/4	1/8

Profile-most probable k -mer in a sequence: the k -mer with the highest

$\Pr(k\text{-mer} \mid \text{Profile})$
among all k -mers in this sequence.

6-mer	Prob(6-mer Profile)	
CTATAAACCTTACAT	$1/8 \times 1/8 \times 3/8 \times 0 \times 1/8 \times 0$	0
CTATAAACCTTACAT	$1/2 \times 7/8 \times 0 \times 0 \times 1/8 \times 0$	0
CTATAAACCTTACAT	$1/2 \times 1/8 \times 3/8 \times 0 \times 1/8 \times 0$	0
CTATAAACCTTACAT	$1/8 \times 7/8 \times 3/8 \times 0 \times 3/8 \times 0$	0
CTATAAACCTTACAT	$1/2 \times 7/8 \times 3/8 \times 5/8 \times 3/8 \times 7/8$	.0336
CTATAAACCTTACAT	$1/2 \times 7/8 \times 1/2 \times 5/8 \times 1/4 \times 7/8$	.0299
CTATAAACCTTACAT	$1/2 \times 0 \times 1/2 \times 0 \times 1/4 \times 0$	0
CTATAAACCTTACAT	$1/8 \times 0 \times 0 \times 0 \times 0 \times 1/8 \times 0$	0
CTATAAACCTTACAT	$1/8 \times 1/8 \times 0 \times 0 \times 3/8 \times 0$	0
CTATAAACCTTACAT	$1/8 \times 1/8 \times 3/8 \times 5/8 \times 1/8 \times 7/8$	.0004

GreedyMotifSearch

Given a set of k -mers $Motif_1, \dots, Motif_{i-1}$ in the first $i-1$ sequences in Dna :

- form *Profile* from them
- form $Motif_1, \dots, Motif_{i-1}, Motif_i$ by adding the *Profile*-most probable k -mer $Motif_i$ in the i -th sequence to the growing set of k -mers

ITERATE

Outline

- From Implanted Patterns to Regulatory Motifs
 - Implanting Patterns into Strings
 - Do We Have a Clock Gene?
 - Implanted Motif Problem
 - Motif Finding Problem (Music Break)
 - Median String Problem
 - Greedy Motif Search
- **How Rolling Dice Helps Us Find Regulatory Motifs**
 - **Randomized Motif Search**
 - How do Bacteria Hibernate?
 - Gibbs Sampling
 - Pseudocounts

From Motifs to Profile to Motifs to Profile to Motifs...

Given *Motifs*, we can construct

Profile(*Motifs*)

Given an arbitrary *Profile* and a set of sequences *Dna*, we can construct

Motifs(*Profile*, *Dna*)

as a set of *Profile*-most probable *k*-mers in each sequence from *Dna*.

Iterate!

Motifs(*Profile*(*Motifs*(*Profile*(*Motifs*), *Dna*)), *Dna*), *Dna*)...



RandomizedMotifSearch

RandomizedMotifSearch(*Dna*, k , t)

randomly select k -mers $Motifs = (Motif_1, \dots, Motif_t)$ in each string from *DNA*

$bestMotifs \leftarrow Motifs$

while forever

$Profile \leftarrow Profile(Motifs)$

$Motifs \leftarrow Motifs(Profile, Dna)$

if $Score(Motifs) < Score(bestMotifs)$

$bestMotifs \leftarrow Motifs$

else

return($bestMotifs$)

How in the World Can This Algorithm Find Anything?

When we form k -mers *randomly*, it results in a **uniform** expected *Profile* with every entry $1/4$. Such a uniform profile is useless because it does not provide any clues about the implanted motif.

RandomizedMotifSearch in Action

Dna with implanted
(4,1)-motif ACGT

```
ttACCTtaac
gATGTctgtc
ccgGCGTtag
cactaACGAg
cgtcagAGGT
```

Randomly select
Motifs (in bold)

```
ttACCTtaac
gATGTctgtc
ccgGCGTtag
cactaACGAg
cgtcagAGGT
```

Motifs

```
t a a c
G T c t
c c g G
a c t a
A G G T
```

Profile(*Motifs*)

A:	2/5	1/5	1/5	1/5
C:	1/5	2/5	1/5	1/5
G:	1/5	1/5	2/5	1/5
T:	1/5	1/5	1/5	2/5

```
.0016/ttAC .0016/tACC .0128/ACCT .0064/CCTt .0016/Ctta .0016/Ttaa .0016/taac
.0016/gATG .0128/ATGT .0016/TGTc .0032/GTct .0032/Tctg .0032/ctgt .0016/tgtc
.0064/ccgG .0036/cgGC .0016/gGCG .0128/GCGT .0032/CGTt .0016/Gtta .0016/Ttag
.0032/cact .0064/acta .0016/ctaA .0016/taAC .0032/aACG .0128/ACGA .0016/CGAg
.0016/cgtc .0016/gtca .0016/tcag .0032/cagA .0032/agAG .0032/gAGG .0128/AGGT
```

```
ttACCTtaac
gATGTctgtc
ccgGCGTtag
cactaACGAg
cgtcagAGGT
```

Motifs (*Profile* (*Motifs*), *Dna*)

How in the World Can This Algorithm Find Anything?

When we select k -mers *randomly*, it results in a **uniform expected Profile** where every entry is $1/4$. Such a uniform profile is useless because it does not provide any clues about the implanted motif.

But strings in **Dna** are not truly random since they include implanted motifs!

These implanted motifs result in a **biased expected Profile**.

Where does this statistical bias point to?

RandomizedMotifSearch in Action

DNA with implanted
(4,1)-motif **ACGT**

```
ttACCTtaac
gATGTctgtc
ccgGCGTtag
cactaACGAg
cgtcagAGGT
```

Randomly select
Motifs (in bold)

```
ttACCTtaac
gATGTctgtc
ccgGCGTtag
cactaAGGT
cgtcagAGGT
```

Motifs

```
t  a  a  c
G T  c  t
c  c  g  G
a  c  t  a
A G G T
```

Profile(**Motifs**)

A:	2/5	1/5	1/5	1/5
C:	1/5	2/5	1/5	1/5
G:	1/5	1/5	2/5	1/5
T:	1/5	1/5	1/5	2/5

The statistical bias points in the direction of the implanted (4,1)-motif **ACGT** because one of the randomly selected 4-mers happened to be the implanted pattern **AGGT** simply by chance.

Outline

- **From Implanted Patterns to Regulatory Motifs**
 - Implanting Patterns into Strings
 - Do We Have a Clock Gene?
 - Implanted Motif Problem
 - Motif Finding Problem (Music Break)
 - Median String Problem
 - Greedy Motif Search
- How Rolling Dice Helps Us Find Regulatory Motifs
 - Randomized Motif Search
 - **How do Bacteria Hibernate?**
 - Gibbs Sampling
 - Pseudocounts

How Do you test if you have Tuberculosis?

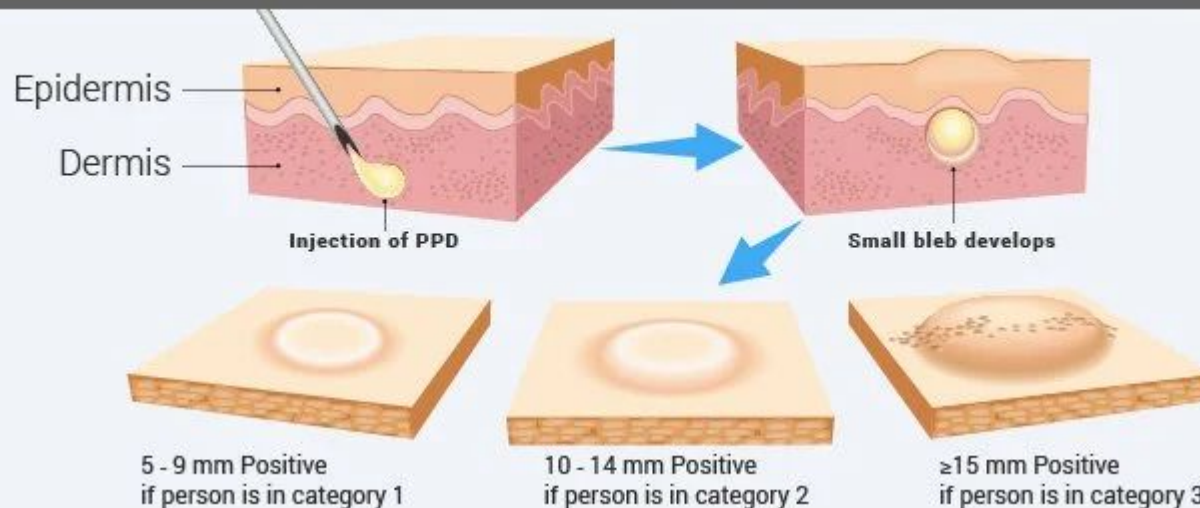


Mantoux test is a tool for screening for tuberculosis (TB) and for tuberculosis diagnosis.

Purified protein derivative (PPD) tuberculin is a precipitate of species-nonspecific molecules obtained from cultures

A person who has been exposed to the bacteria is expected to mount an immune response in the skin containing the bacterial proteins.

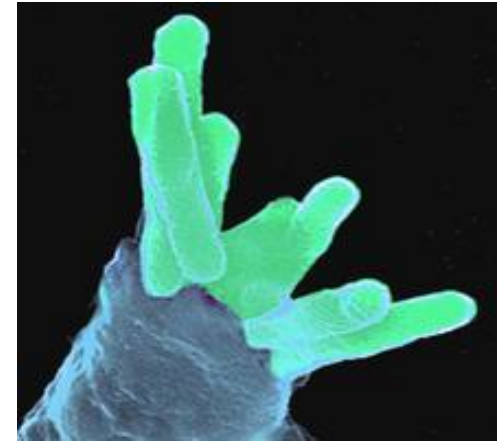
Purified Protein Derivative (PPD) Test | Method of Detecting Tuberculosis



How Do Tuberculosis Bacteria Hibernate to Hide from Antibiotics?

Tuberculosis (TB) 2.2 million cases in India.
26% of cases and 38% deaths in the world

TB is caused by *Mycobacterium tuberculosis* (MTB). MTB strains that resist all antibiotics are now emerging.



MTB can persist for decades without causing disease:
1/3rd of the world's population carry latent MTB.

How does MTB switch from a latent to active state?

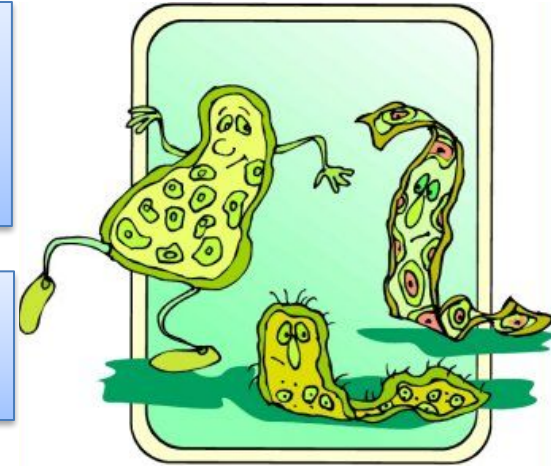
TB and Dormancy Survival Regulator (DosR)

“Hibernating” MTB forms metabolically dormant **spores** that can survive in tough conditions (e.g., oxygen shortage).

Oxygen shortage (**hypoxia**) is associated with latent forms of TB.

Biologists found 25 genes whose expression (how many RNA molecules they make > how many proteins they make) significantly changes in hypoxia. They conjectured that expression of these genes is controlled by a transcription factor that was named **Dormancy Survival Regulator (DosR)**.

Can **MedianString** and **RandomizedMotifSearch** discover a regulatory motif that controls the expression of these genes?



Analyzing Dormancy Survival Regulator

We have no idea what the length of a motif (k) is.

MedianString			RandomizedMotifSearch		
		score			score
$k=8$:	CATCGGCC	11	CCGACGGG		13
$k=9$:	GGCGGGGAC	16	CCATCGGCC		16
$k=10$:	GGTGGCCACC	19	CCATCGGCCC		21
$k=11$:	GGACTTCCGGC	20	ACCTTCGGCCC		25
$k=12$:	GGAC TT CGGCC	23	GGAC CAA CGGCC		28

Length $k = 20$ motif returned by **RandomizedMotifSearch**:

CGGGGAC**CTA**CGG**CCC** TAGC

How well our predictions match the real binding sites?

Entropy of a Motif

Profile

A:	.2	.2	0	0	0	0	.9	.1	.1	.1	.3	0
C:	.1	.6	0	0	0	0	0	.4	.1	.2	.4	.6
G:	0	0	1	1	.9	.9	.1	0	0	0	0	0
T:	.7	.2	0	0	.1	.1	0	.5	.8	.7	.3	.4

Every column of the profile matrix is a **probability distribution**.

Entropy of a probability distribution $(p_1, \dots, p_n)$ is

$$-\sum p_j \cdot \log_2(p_j)$$

through all probabilities



Entropy of **uniform** distribution $(\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4})$ is $-4 \cdot \frac{1}{4} \log_2(\frac{1}{4}) = \mathbf{2}$ (**maximal entropy**).

Entropy of a **completely conserved** column like $(\mathbf{1}, \mathbf{0}, \mathbf{0}, \mathbf{0})$ is $\mathbf{0}$ (**minimal entropy**).

The entropy varies from **0** to **2**, entropy of the 1st column (0.2, 0.1, 0.0, 0.7):

$$-(0.2 \cdot \log_2 0.2 + 0.1 \cdot \log_2 0.1 + 0.0 \cdot \log_2 0.0 + 0.7 \cdot \log_2 0.7) \approx 1.147$$

Entropy of a motif – sum of entropies of columns of its profile matrix.

Motif Logo

	A:	.2	.2	0	0	0	0	.9	.1	.1	.1	.3	0
	C:	.1	.6	0	0	0	0	0	.4	.1	.2	.4	.6
<i>Profile(Motifs)</i>	G:	0	0	1	1	.9	.9	.1	0	0	0	0	0
0	T:	.7	.2	0	0	.1	.1	0	.5	.8	.7	.3	.4
<i>Consensus(Motifs)</i>		T	C	G	G	G	G	A	T	T	T	C	
C													

motif
logo



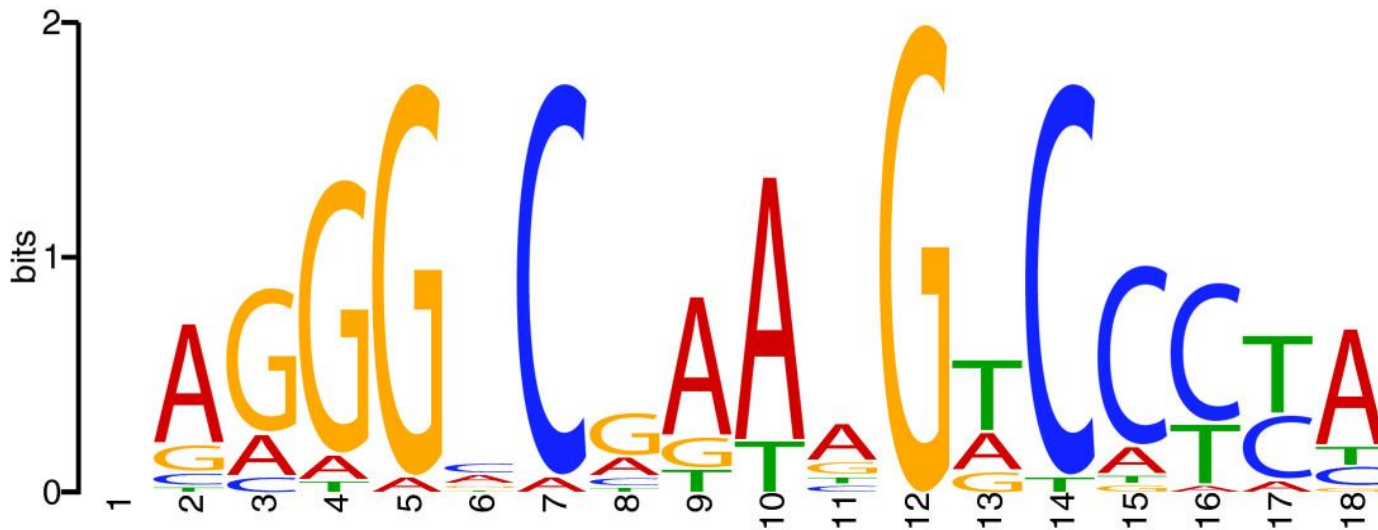
The **total height** of 4 nucleotides in each position: **2 - entropy**.

The **relative sizes** of 4 nucleotides in each position: frequencies in the column.

Predicted vs. Real Dormancy Survival Regulator Binding Site

Length 20 motif returned by **RandomizedMotifSearch**:

CGGGGGACCTACGGCCCTAGC



Motif logo of real Dormancy Survival Regulator binding sites

Outline

- **From Implanted Patterns to Regulatory Motifs**
 - Implanting Patterns into Strings
 - Do We Have a Clock Gene?
 - Implanted Motif Problem
 - Motif Finding Problem (Music Break)
 - Median String Problem
 - Greedy Motif Search
- How Rolling Dice Helps Us Find Regulatory Motifs
 - Randomized Motif Search
 - How do Bacteria Hibernate?
 - **Gibbs Sampling**
 - Pseudocounts

Gibbs Sampling

RandomizedMotifSearch may replace all k -mers in a single iteration and thus may potentially discard a nearly correct motif.

Gibbs sampling replaces a single k -mer at each iteration and thus moves with more caution in the space of all motifs.

ttacctt aac		t tac cttaac
g ata tctgtc		gat atc tgtc
acg gcgttcg	→	acggcgt ttc g
ccct aaa gag		ccctaa aga g
cgtc aga ggt		cgt cagagggt

RandomizedMotifSearch
(may change **all** k -mers in 1 iteration)

ttacctt aac		ttacctt aac
g ata tctgtc		gatatc tgtc
acg gcgttcg	→	acg gcgttcg
ccct aaa gag		ccct aaa gag
cgtc aga ggt		cgtc aga ggt

GibbsSampler
(changes a **single** k -mer in 1 iteration)

GibbsSampler in Action

Randomly select
Motifs (in bold)

ttACCT**taac**
gAT**GT**ctgtc
ccg**GCGT**tag
cactaACGA**g**
cgtcag**AGGT**

Randomly remove
a *k*-mer in **Motifs**

ttACCT**taac**
gAT**GT**ctgtc

cactaACGA**g**
cgtcag**AGGT**

Motifs matrix

t	a	a	c
G	T	c	t

a	c	t	a
A	G	G	T

Choose a new st
in the deleted se
on rolling

ITERATE

Calculate the probabilities
of all *k*-mers in the deleted
string **ccgGCGTtag**

Profile matrix

A:	2	1	1	1
C:	0	1	1	1
G:	1	1	1	0
T:	1	1	1	2

A:	2/4	1/4	1/4	1/4
C:	0	1/4	1/4	1/4
G:	1/4	1/4	1/4	0
T:	1/4	1/4	1/4	2/4

0 (ccgG) 0 (cgGC) 0 (gGCG) **1/128 (GCGT)** 0 (CGTt) **1/256 (GTta)** 0 (Ttag)

Roll a 7-sided die with probabilities of sides proportional to probabilities of *k*-mers

Gibbs Sampler Outline

1. Form *Motifs* by randomly selecting a k -mer in each sequence
2. Randomly choose one of selected k -mers (from *RemovedSequence*) and remove it from *Motifs*.
3. Create *Profile* from the remaining k -mers in *Motifs*.
4. For each k -mer in *RemovedSequence*, calculate $Pr(k\text{-mer} / \text{Profile})$ resulting in $n-k+1$ probabilities:
$$p_1, p_2, \dots, p_{n-k+1}.$$
5. Roll a die (with $n-k+1$ sides) where probability of ending up at side i is proportional to p_i .
6. Choose a new starting position based on rolling the die. Add the k -mer starting at this position in *RemovedSequence* to *Motifs*.
7. Repeat steps 2-6.

Gibbs Sampler in Action

Randomly select
Motifs (in bold)

ttACCT**taac**
gAT**GT**ctgtc
ccgGCGTtag
cactaACGA**g**
cgtcag**AGGT**

Randomly remove
a *k*-mer in *Motifs*

ttACCT**taac**
gAT**GT**ctgtc

cactaACGA**g**
cgtcag**AGGT**

Motifs matrix

t	a	a	c
G	T	c	t

a	c	t	a
A	G	G	T

Count matrix

A:	2	1	1	1
C:	0	1	1	1
G:	1	1	1	0
T:	1	1	1	2

Calculate the probabilities
of all *k*-mers in the deleted
string **ccgGCGT**tag

Profile matrix

A:	2/4	1/4	1/4	1/4
C:	0	1/4	1/4	1/4
G:	1/4	1/4	1/4	0
T:	1/4	1/4	1/4	2/4

0 (ccgG) **0** (cgGC) **0** (gGCG) **1/128** (GCGT) **0** (CGTt) **1/256** (GTta) **0** (Ttag)

Should we roll a two-sided die???



Cromwell's Rule: Never say never



Cromwell's rule: the use of probabilities of 0 or 1 should be avoided.

We should leave a little probability that the sun will not rise tomorrow.

Outline

- **From Implanted Patterns to Regulatory Motifs**
 - Implanting Patterns into Strings
 - Do We Have a Clock Gene?
 - Implanted Motif Problem
 - Motif Finding Problem (Music Break)
 - Median String Problem
 - Greedy Motif Search
- How Rolling Dice Helps Us Find Regulatory Motifs
 - Randomized Motif Search
 - How do Bacteria Hibernate?
 - Gibbs Sampling
 - **Pseudocounts**

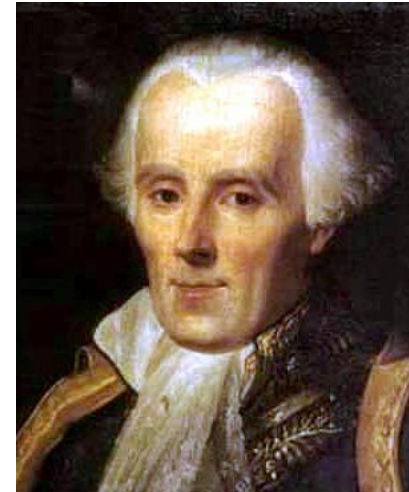
Laplace's Rule of Succession

In small datasets, there is always a chance that a possible event does not occur (e.g., **zeroes** in *Count* matrix).

Randomized algorithms do not like zeroes and introduce **pseudocounts** that inflate the probabilities of rare events and eliminate empirical zero-frequencies.

Laplace calculated the probability that the sun will not rise tomorrow, given that it has risen every day for the past 5000 years (1 in 1826251)

His estimate was ridiculed because his opponents failed to understand its importance.

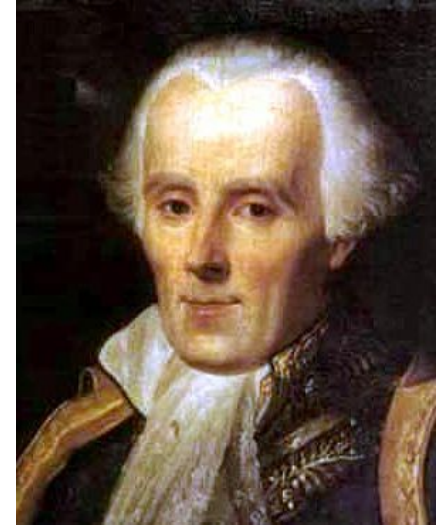


If we repeat an experiment that can result in a success or failure **n** times and get **s** successes, what is the probability that the next repetition will be a success?

Pseudocounts

If $X_1, \dots, X_{n+1}$ are *conditionally* independent random boolean variables (failure 0 or success 1) then:

$$\Pr(X_{n+1}=1|X_1+\dots+X_n=s) = (s+1)/(n+2)$$



Since we have the prior knowledge that both success and failure are possible, we essentially made $n+2$ rather than n observations since we (implicitly) observed one success and one failure before we even started the experiments.

Thus, we have made $n+2$ observations (known as **pseudocounts**) with $s+1$ successes.

What would Laplace Do to Get Rid of **Zeroes**?

ttACCTtaac
gATGTctgtc
ccgGCGTtag
cactaACGA
cgtcagAGGT

ttACCTtaac
gATGTctgtc

cactaACGA
cgtcagAGGT

t	a	a	c
G	T	c	t

a	c	t	a
A	G	G	T

Choose a new starting position
in the deleted sequence based
on rolling the die

Update
Count Matrix with
pseudocounts

A:	2+1	1+1	1+1	1+1
C:	0+1	1+1	1+1	1+1
G:	1+1	1+1	1+1	0+1
T:	1+1	1+1	1+1	2+1

Recalculate the probabilities
of all 4-mers in the deleted
string ccgGCGTtag

Update
Profile Matrix

A:	3/8	2/8	2/8	2/8
C:	1/8	2/8	2/8	2/8
G:	2/8	2/8	2/8	1/8
T:	2/8	2/8	2/8	3/8

1/1024 (ccgG) 2/1024 (cgGC) 2/1024 (gGCG) **6/1024 (GCGT)** 3/1024 (CGTt) 4/1024 (GTta) 2/1024 (Ttag)

Roll a 7-sided rather than 2-sided die!



Gibbs Sampler Continues

Randomly select
Motifs (in bold)

```
ttACCTtaac
gATGTctgtc
ccgGCGTtag
cactaACGAg
cgtcagAGGT
```

Randomly remove
a *k*-mer in **Motifs**

```
-----
gATGTctgtc
ccgGCGTtag
cactaACGAg
cgtcagAGGT
```

Motifs matrix

```
-----
G    T    c    t
G    C    G    T
a    c    t    a
A    G    G    T
```

Choose a new starting position
in the deleted sequence based
on rolling the die

Pseudocount Matrix

```
A:  2+1  0+1  0+1  1+1
C:  0+1  2+1  1+1  0+1
G:  2+1  1+1  2+1  0+1
T:  0+1  1+1  1+1  3+1
```

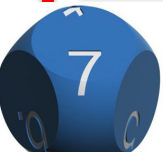
Calculate the probabilities
of all *k*-mers in the deleted
string tt**ACCT**taac

Construct *Profile*
using Laplace's
Rule

```
A:  3/8  1/8  1/8  2/8
C:  1/8  3/8  2/8  1/8
G:  3/8  2/8  3/8  1/8
T:  1/8  2/8  2/8  4/8
```

2/4096 (tt**AC**) 2/4096 (t**ACC**) 72/4096 (**ACCT**) 24/4096 (**CCT**t) 8/4096 (C**T**ta) 4/4096 (**T**taa) 1/4096 (taac)

Roll a 7-sided die



Gibbs Sampler Continues and Continues

Randomly select
Motifs (in bold)

```
ttACCTtaac
gATGTctgtc
ccgGCGTtag
cactaACGAg
cgtcagAGGT
```

Randomly remove
a *k*-mer in *Motifs*

```
ttACCTtaac
gATGTctgtc
ccgGCGTtag
-----
cgtcagAGGT
```

Motifs matrix

A	C	C	T
G	T	c	t
G	C	G	T

A	G	G	T

Choose a new starting position
in the deleted sequence based
on rolling the die

Pseudocount matrix

A:	2+1	0+1	0+1	0+1
C:	0+1	2+1	2+1	0+1
G:	2+1	1+1	2+1	0+1
T:	0+1	1+1	0+1	4+1

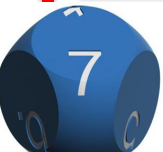
Calculate the probabilities
of all *k*-mers in the deleted
string tt**ACCT**taac

Create *Profile*
using Laplace's
Rule

A:	3/8	1/8	1/8	1/8
C:	1/8	3/8	3/8	1/8
G:	3/8	2/8	3/8	1/8
T:	1/8	2/8	1/8	5/8

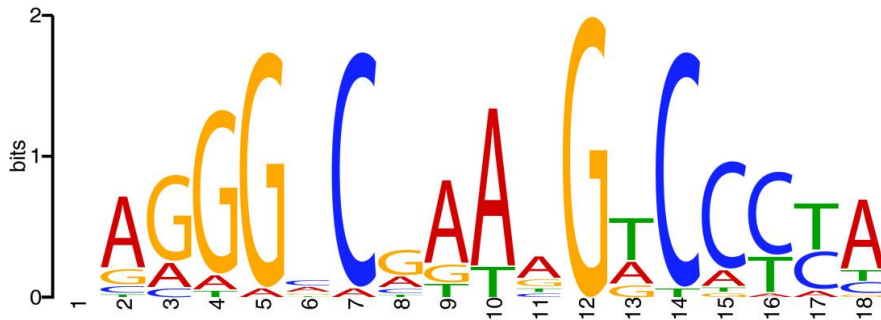
2/4096 (tt**AC**) 2/4096 (t**ACC**) 72/4096 (**ACCT**) 24/4096 (**CCT**t) 8/4096 (C**T**ta) 4/4096 (**T**taa) 1/4096 (taac)

Roll a 7-sided die



The Challenge of Predicting Degenerative Motifs

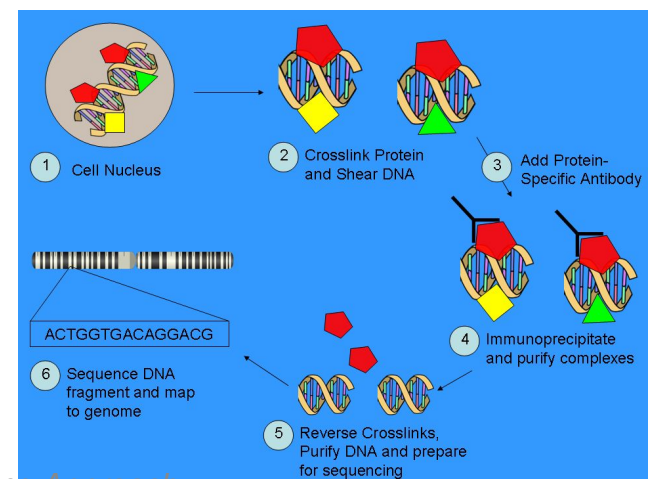
CGGGGACCTACGGCCCTAG



Even the best motif finding algorithms fail to find some highly degenerative motifs, e.g., (15,5)-motifs are nearly impossible to find if they are randomly inserted in **1000**-nucleotide long random strings.

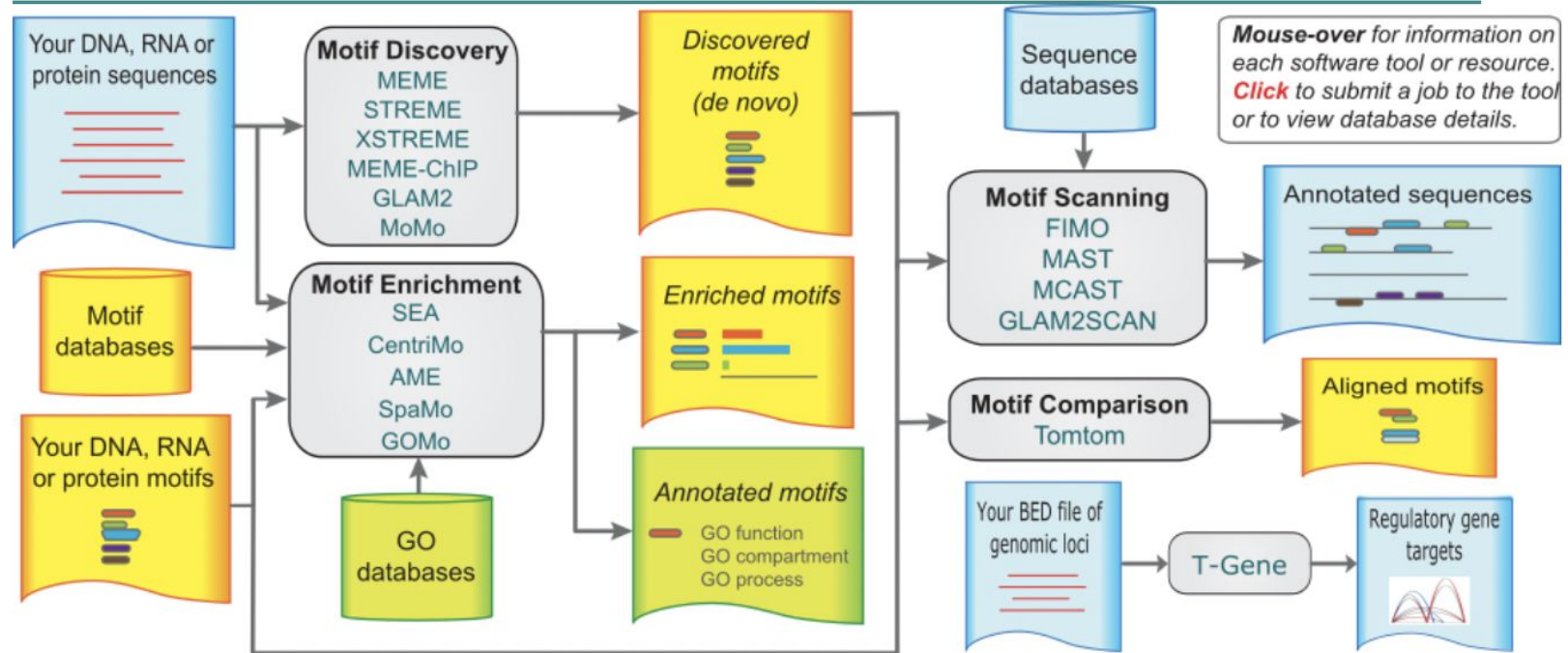
We can still solve the Implanted Motif Problem but it will find irrelevant 15-mers

ChIP Sequencing provides an experimental solution to this problem by reducing the length of sequences to search for motifs. For example, if (15,5)-motif is inserted in **100**-nucleotide (rather than **1000**) long random strings, it will be easier to find!



The MEME Suite

Motif-based sequence analysis tools



MEME

Multiple Em for Motif Elicitation



SEA

Simple Enrichment Analysis



FIMO

Find Individual Motif Occurrences

Improved



STREME

Sensitive, Thorough, Rapid, Enriched Motif Elicitation



AME

Analysis of Motif Enrichment



MAST

Motif Alignment & Search Tool



XSTREME

Motif Discovery and Enrichment Analysis



CentriMo

Local Motif Enrichment Analysis



MCAST

Motif Cluster Alignment and Search Tool



MEME-ChIP

Motif Analysis of Large Nucleotide Datasets



SpaMo

Spaced Motif Analysis Tool

Improved



GLAM2Scan

Scanning with Gapped Motifs